Practical Training Handout: Bacterial Genomic AMR Surveillance

Introduction

Purpose of the training

This handout guides professionals through a practical bacterial whole-genome sequencing workflow for antimicrobial resistance (AMR) surveillance. Participants will learn how to organise data, assess sequence quality, trim reads, assemble genomes, call variants, check assembly quality, detect AMR genes, and build phylogenies. The training is intentionally practical: each module includes commands, expected outputs, quality checkpoints, and reporting notes.

Audience

The handout is written for laboratory scientists, public health professionals, research assistants, postgraduate students, and early-career bioinformaticians who are new to Linux-based bacterial genomics but need a professional surveillance workflow.

Important training assumptions

Commands rely on consistent file naming and shell variables. Example filenames use SAMPLE_R1.fastq.gz, SAMPLE_R2.fastq.gz for Illumina paired-end reads and SAMPLE_ONT.fastq.gz for Oxford Nanopore reads. Replace SAMPLE with your own isolate ID.

Core surveillance idea

AMR genomic surveillance links sequence data to public health interpretation. The raw reads are not the final result. The goal is a reproducible chain of evidence: raw data quality, cleaned reads, assembly or mapping outputs, contamination/completeness checks, AMR gene calls, variant calls, phylogenetic context, and a clear report of software and database versions.

Professional practice

Keep commands in a log file, store outputs in clearly named folders, never overwrite raw data, report software and database versions, and treat AMR gene calls as genomic predictions that may require phenotypic confirmation and expert interpretation.

Key points for the complete training

- Work reproducibly from a home-directory project folder.
- Keep raw reads unchanged and document all derived outputs.
- Assess quality before and after read cleaning.
- Use assembly-based and reference-based analyses for complementary questions.
- Report software versions, database versions and quality limitations with every result.

Suggested training flow

Stage	Activity
pre-training	Install Miniforge/conda, create environments, download test reads and references, confirm internet access for database downloads.
Session 1	Linux basics, folder structure, file inspection, compression, checksums, and command logging.
Session 2	Read QC using FastQC/MultiQC for Illumina data and

	NanoPlot/NanoQC for ONT data.
Session 3	Read trimming using fastp and Trimmomatic, then repeat QC to compare before and after trimming.
Session 4	De novo assembly: Unicycler for Illumina/hybrid assembly and Flye for long-read assembly.
Session 5	Reference-based mapping and variant calling using BWA/minimap2, SAMtools and BCFtools.
Session 6	rMAP pipeline demonstration as an example of automated, reproducible bacterial WGS analysis.
Session 7	Assembly QC using QUAST and CheckM; decide whether assemblies are suitable for downstream AMR and phylogenetic analyses.
Session 8	AMR detection using AMRFinderPlus and ABRicate; compare databases and reporting formats.
Session 9	Phylogenetics with BactInspector, Snippy and IQ-TREE; interpret tree outputs cautiously.

Quick output map

Folder	Purpose
raw_reads/	Original FASTQ files; do not edit or overwrite.
qc_raw/	FastQC, MultiQC, NanoPlot and NanoQC reports before trimming.
trimmed_reads/	Cleaned reads from fastp or Trimmomatic.
assemblies/	Unicycler/Flye output assemblies.
mapping/ and variants/	BAM files, mapping summaries and VCF outputs.
assembly_qc/	QUAST and CheckM reports.
amr/	AMRFinderPlus and ABRicate results.
phylogeny/	Snippy alignments and IQ-TREE outputs.
logs/	Command logs, software versions and database reports.

Training Setup, Folder Structure and Environment Preparation

Module introduction

This opening module sets up the practical working environment that participants will use throughout the training. A good folder structure, consistent filenames, conda environments and version logs make the workflow easier to repeat, audit and teach. Participants should complete this section before running any analysis module.

Key points

- Work inside a dedicated project folder under the home directory.
- Keep raw reads unchanged and store derived outputs in separate folders.
- Use consistent filenames.
- Use separate conda environments to reduce software conflicts.
- Record tool versions, database versions and important command logs from the start.

Learning outcomes

- Create a reproducible home-directory project structure.
- Install Miniforge/conda and mamba.
- Create separate conda environments for major analysis tasks.
- Record software and database versions for reporting.

Practical steps

Recommended project structure

Create a project folder in your home directory. Do not work directly inside Downloads or Desktop.

```
cd ~
mkdir -p
AMR_training/{raw_reads,refs,metadata,qc_raw,qc_trimmed,trimmed_reads,assemblies,mapping,variants,rma
p_demo,assembly_qc,amr,phylogeny,logs,db,reports,scripts}
cd ~/AMR_training
```

Expected output: running `ls` should show folders such as raw_reads, refs, qc_raw, trimmed_reads, assemblies, mapping, variants, amr and phylogeny.

Example filenames

Illumina paired-end reads:
- raw_reads/KPN001_R1.fastq.gz
- raw_reads/KPN001_R2.fastq.gz

Nanopore reads:
- raw_reads/KPN001_ONT.fastq.gz

Reference genome:
- refs/reference.fasta

Install Miniforge/conda

Miniforge is recommended because it works well with conda-forge and bioconda.

```
cd ~
mkdir -p ~/software
cd ~/software
curl -L -O https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-
x86_64.sh
bash Miniforge3-Linux-x86_64.sh
```

Close and reopen the terminal

Test installation:

```
conda --version
```

Expected output: version number for conda.

Configure conda channels

Use strict channel priority for reproducible bioinformatics installations.

```
conda config --add channels conda-forge
conda config --add channels bioconda
conda config --set channel_priority strict
conda config --show channels
```

Expected output: conda-forge and bioconda should appear in the channel list.

Create training environments

A single large environment can become difficult to solve. For training, create task-specific environments.

```
# General QC and trimming
conda create -y -n amr_qc fastqc multiqc fastp trimmomatic nanoplot nanoqc seqkit pigz

# Assembly
conda create -y -n amr_assembly unicycler flye quast
```

```
# Mapping and variants
conda create -y -n amr_mapping bwa minimap2 samtools bcftools snippy iqtree

# AMR detection
conda create -y -n amr_amr ncbi-amrfinderplus abricate

# CheckM often has heavier dependencies; install separately
conda create -y -n amr_checkm checkm-genome
```

If a package fails to install, record the error and use the instructor-provided assistance or container image.

Version-report template

Run these commands whenever you begin or finish an analysis. Save the output in the logs folder.

```
mkdir -p logs; { date; uname -a; conda info --envs; fastqc --version 2>/dev/null || true; multiqc --version 2>/dev/null || true; fastp --version 2>/dev/null || true; trimmomatic -version 2>/dev/null || true; unicycler --version 2>/dev/null || true; flye --version 2>/dev/null || true; bwa 2>&1 | head -n 3 || true; minimap2 --version 2>/dev/null || true; samtools --version | head -n 2 2>/dev/null || true; bcftools --version | head -n 2 2>/dev/null || true; quast.py --version 2>/dev/null || true; checkm version 2>/dev/null || true; amrfinder --version 2>/dev/null || true; abricate --version 2>/dev/null || true; snippy --version 2>/dev/null || true; iqtree2 --version 2>/dev/null || iqtree --version 2>/dev/null || true; } | tee logs/software_versions.txt
```

Expected output: a text file named logs/software_versions.txt containing software versions and system information.

End-of-module checklist

- Commands ran without unresolved errors.
- Expected output files were created.
- Relevant logs were saved in the logs folder.
- Software and database versions were recorded where applicable.
- Any quality warnings or failed samples were documented.

Module 1: Linux Basics for Bacterial Genomics

Module introduction

Linux is the foundation of most bacterial genomics workflows. This module introduces only the commands needed to move confidently through files, inspect sequence data, run tools and keep a reproducible record of work. The goal is not to become a system administrator, but to become comfortable enough to run and troubleshoot common surveillance analyses.

Key points

- Use pwd, ls, cd and tree/find to understand where you are working.
- Use zcat/head to inspect compressed FASTQ files without decompressing them.
- Use variables and loops to process multiple samples consistently.
- Use redirection and tee to save command outputs and logs.
- File extensions matter: FASTQ, FASTA, BAM, VCF and TSV represent different data types.

Learning outcomes

- Navigate directories and inspect files.
- Use pipes, wildcards, redirection and loops.
- Understand FASTQ/FASTA/GFF/VCF/BAM file types.
- Create command logs for reproducibility.

Practical steps

Key file types

FASTQ stores sequencing reads and quality scores. FASTA stores reference genomes, assemblies or contigs. GFF/GTF stores genomic feature annotations. SAM/BAM/CRAM store read alignments. VCF/BCF stores variants. TSV/CSV stores tabular results. HTML reports are common for QC and MultiQC summaries.

Navigation and inspection commands

```
pwd
ls -lh
cd ~/AMR_training
tree -L 2 . 2>/dev/null || find . -maxdepth 2 -type d

# View compressed FASTQ without decompressing
zcat raw_reads/KPN001_R1.fastq.gz | head -n 8

# Count reads: four lines per FASTQ record
zcat raw_reads/KPN001_R1.fastq.gz | wc -l
```

Expected output: `pwd` shows your current folder, `ls -lh` shows readable file sizes, and the first 8 FASTQ lines represent 2 reads.

Wildcards, variables and loops

```
cd ~/AMR_training

# List all R1 files
ls raw_reads/*R1_001.fastq.gz

# Loop over paired-end samples
for r1 in raw_reads/*R1_001.fastq.gz; do
  sample=$(basename "$r1" _R1_001.fastq.gz)
  r2="raw_reads/${sample}R2_001.fastq.gz"
  echo "$sample"
  echo "R1=$r1"
  echo "R2=$r2"
done

# Loop over paired-end samples (one-liner)

for r1 in raw_reads/*R1_001.fastq.gz; do sample=$(basename "$r1" _R1_001.fastq.gz);
r2="raw_reads/${sample}R2_001.fastq.gz"; echo "$sample"; echo "R1=$r1"; echo "R2=$r2"; done
```

Expected output: sample IDs and corresponding R1/R2 paths.

Redirection and logging

```
mkdir -p logs

# Save a command output
ls -lh raw_reads > logs/raw_read_files.txt

# Save both screen output and a log file
fastqc --version 2>&1 | tee logs/fastqc_version.txt

# Append instead of overwrite
date >> logs/analysis_notes.txt
```

Professional tip: keep one notes file for decisions such as excluded samples, failed commands, and thresholds used.

End-of-module checklist

- Commands ran without unresolved errors.
- Expected output files were created.
- Relevant logs were saved in the logs folder.

- Software and database versions were recorded where applicable.
- Any quality warnings or failed samples were documented.

Module 2: Read QC with FastQC, MultiQC, NanoPlot and NanoQC

Module introduction

Read quality control is the first scientific checkpoint after sequencing. Poor-quality reads, adapter contamination or unusual length/GC patterns can affect trimming, assembly, mapping, AMR detection and phylogenetic interpretation. This module teaches participants how to generate and interpret short-read and long-read QC reports before proceeding.

Key points

- Run QC before trimming or assembly.
- Use FastQC plus MultiQC for Illumina short reads.
- Use NanoPlot and NanoQC for Oxford Nanopore reads.
- Review total bases, quality distributions, adapter content, GC content and read length distribution.
- Do not continue blindly if QC suggests contamination, very low coverage or wrong files.

Learning outcomes

- Run FastQC and MultiQC on Illumina reads.
- Run NanoPlot and NanoQC on long-read data.
- Identify common QC warnings and decide whether trimming or resequencing is needed.
- Save QC summaries for reports.

Practical steps

Illumina QC with FastQC

```
conda activate amr_qc
cd ~/AMR_training
mkdir -p qc_raw/fastqc logs

fastqc raw_reads/*_R1.fastq.gz raw_reads/*_R2.fastq.gz --outdir qc_raw/fastqc -threads 4 2>&1 | tee
logs/fastqc_raw.log
```

Expected outputs: one `.html` and one `.zip` report per FASTQ file in qc_raw/fastqc.

Summarise Illumina QC with MultiQC

```
multiqc qc_raw/fastqc -o qc_raw/multiqc_raw 2>&1 | tee logs/multiqc_raw.log
```

Expected outputs: qc_raw/multiqc_raw/multiqc_report.html and a multiqc_data folder. Open the HTML report in a browser.

Merge Nanopore Fragments

```
dir="raw_reads/KPN001_ONT"; cat "$dir"/*.fastq.gz > "$dir/$(basename "$dir").fastq.gz"
```

Nanopore QC with NanoPlot

```
mkdir -p qc_raw/nanoplot
NanoPlot --fastq raw_reads/KPN001_ONT/KPN001_ONT.fastq.gz --outdir qc_raw/nanoplot/KPN001 --threads 4
--loglength 2>&1 | tee logs/nanoplot_KPN001.log
```

Expected outputs: NanoPlot-report.html, length histograms, quality plots and summary statistics.

Nanopore QC with NanoQC

```
mkdir -p qc_raw/nanoqc
nanoQC raw_reads/KPN001_ONT.fastq.gz -o qc_raw/nanoqc/KPN001 2>&1 | tee logs/nanoqc_KPN001.log
```

Expected outputs: an HTML QC report and supporting plots for the long-read FASTQ.

QC interpretation checklist

Check: total reads, per-base quality, adapter content, sequence length distribution, GC distribution, duplication, overrepresented sequences, N content, and whether read lengths match the sequencing method. For ONT reads, pay attention to N50 read length, total bases, read quality distribution and unusually short read enrichment. Document any sample that fails QC before proceeding.

End-of-module checklist

- Commands ran without unresolved errors.
- Expected output files were created.
- Relevant logs were saved in the logs folder.
- Software and database versions were recorded where applicable.
- Any quality warnings or failed samples were documented.

Module 3: Trimming with fastp and Trimmomatic

Module introduction

Trimming removes adapters and low-quality bases so that downstream tools receive cleaner input reads. The module uses fastp and Trimmomatic because both are widely used in bacterial genomics training and surveillance workflows. Participants will learn to compare pre- and post-trimming QC rather than assuming trimming always improves the data.

Key points

- fastp is fast, convenient and produces HTML/JSON reports.
- Trimmomatic is a useful alternative and remains common in older workflows.
- Always compare raw and trimmed QC reports.
- Keep paired reads synchronised after trimming.
- Over-trimming can reduce useful data and harm assembly or mapping.

Learning outcomes

- Run fastp for paired-end reads.
- Run Trimmomatic as an alternative trimmer.
- Compare raw and trimmed QC reports.
- Understand how trimming choices affect downstream assembly and mapping.

Practical steps

Paired-end Illumina reads trimming with fastp

```
conda activate amr_qc
cd ~/AMR_training
mkdir -p trimmed_reads/fastp qc_trimmed/fastp logs

for r1 in raw_reads/*_R1_001.fastq.gz; do
  sample=$(basename "$r1" _R1_001.fastq.gz)
  r2="raw_reads/${sample_R2_001.fastq.gz"
  fastp -i "$r1" -I "$r2" -o trimmed_reads/fastp/${sample}_R1.trim.fastq.gz -O
trimmed_reads/fastp/${sample}_R2.trim.fastq.gz --detect_adapter_for_pe --qualified_quality_phred 20
--length_required 50 --thread 4 --html qc_trimmed/fastp/${sample}.fastp.html --json
qc_trimmed/fastp/${sample}.fastp.json 2>&1 | tee logs/fastp_${sample}.log
done
```

Nanopore raw reads trimming with fastp

```
conda activate amr_qc
```

```
cd ~/AMR_training
mkdir -p trimmed_reads/fastp_nanopore qc_trimmed/fastp_nanopore logs

for r in raw_reads/*.fastq.gz; do

  sample=$(basename "$r" .fastq.gz)

  fastp -i "$r" -o trimmed_reads/fastp_nanopore/${sample}.trim.fastq.gz --qualified_quality_phred 10
--length_required 500 --thread 4 --html qc_trimmed/fastp_nanopore/${sample}.fastp.html --json
qc_trimmed/fastp_nanopore/${sample}.fastp.json 2>&1 | tee logs/fastp_nanopore_${sample}.log

done
```

Expected outputs: trimmed paired FASTQ files, one HTML report and one JSON report per sample.

Paired-end Illumina reads trimming with Trimmomatic

```
mkdir -p trimmed_reads/trimmomatic logs

for r1 in raw_reads/*_R1_001.fastq.gz; do

  sample=$(basename "$r1" _R1_001.fastq.gz)

  r2="raw_reads/${sample}_R2_001.fastq.gz"

  trimmomatic PE -threads 4 -phred33 \

    "$r1" "$r2" \

    trimmed_reads/trimmomatic/${sample}_R1.paired.fastq.gz \

    trimmed_reads/trimmomatic/${sample}_R1.unpaired.fastq.gz \

    trimmed_reads/trimmomatic/${sample}_R2.paired.fastq.gz \

    trimmed_reads/trimmomatic/${sample}_R2.unpaired.fastq.gz \

    ILLUMINACLIP:raw_reads/NexteraPE-PE.fa:2:30:10 \

    SLIDINGWINDOW:4:20 MINLEN:50 \

    2>&1 | tee logs/trimmomatic_${sample}.log

done
```

Expected outputs: paired and unpaired trimmed reads. Use paired reads for most downstream paired-end analyses.

Post-trimming QC for Illumina Paired-End fastp output

```
mkdir -p qc_trimmed/fastqc qc_trimmed/multiqc
fastqc trimmed_reads/fastp/*.fastq.gz --outdir qc_trimmed/fastqc --threads 4
multiqc qc_raw qc_trimmed -o qc_trimmed/multiqc
```

Post-trimming QC for Illumina Paired-End trimmomatic output

```
mkdir -p qc_trimmed/fastqc qc_trimmed/multiqc
fastqc trimmed_reads/trimmomatic/*.fastq.gz --outdir qc_trimmed/fastqc --threads 4
multiqc qc_raw qc_trimmed -o qc_trimmed/multiqc
```

Post-trimming QC for Nanopore read fastp output using NanoQC

```
mkdir -p qc_trimmed/nanoqc
```

```
for fq in trimmed_reads/fastp/*.fastq.gz; do

  sample=$(basename "$fq" .fastq.gz)

  mkdir -p "qc_trimmed/nanoqc/$sample"

  nanoQC "$fq" -o "qc_trimmed/nanoqc/$sample"

done
```

Post-trimming QC for Nanopore read fastp output using Nanoplot

```
mkdir -p qc_trimmed/nanoplot
NanoPlot --fastq trimmed_reads/fastp/*.fastq.gz -o qc_trimmed/nanoplot -t 4
```

Expected output: a combined MultiQC report comparing raw and trimmed reads. Improvements may include reduced adapter content and improved per-base quality at read ends.

Reporting trimming decisions

Report the trimmer name, version, key thresholds, number or percentage of reads retained, and whether paired-end synchronization was preserved. Avoid over-trimming because very short reads can reduce mapping quality and assembly contiguity.

End-of-module checklist

- Commands ran without unresolved errors.
- Expected output files were created.
- Relevant logs were saved in the logs folder.
- Software and database versions were recorded where applicable.
- Any quality warnings or failed samples were documented.

Module 4: De novo Assembly with Unicycler and Flye

Module introduction

De novo assembly reconstructs draft genomes from sequencing reads without relying directly on a reference genome. Assemblies are useful for AMR gene detection, species confirmation, gene content analysis and downstream comparative genomics. This module covers short-read and long-read assembly options appropriate for bacterial isolates.

Key points

- Unicycler is suitable for short-read and hybrid bacterial assembly.
- Flye is suitable for long-read assembly.
- Assembly quality depends strongly on read quality, coverage and contamination.
- Standardise final assembly filenames for downstream tools.
- Assemblies should be checked with QUAST and CheckM before interpretation.

Learning outcomes

- Run Unicycler for Illumina and hybrid assemblies.
- Run Flye for long-read assemblies.
- Recognise common assembly outputs.
- Prepare assemblies for QC and AMR detection.

Practical steps

Short-read assembly with Unicycler

```
conda activate amr_assembly
cd ~/AMR_training
mkdir -p assemblies/unicycler logs

for r1 in trimmed_reads/fastp/*_R1.trim.fastq.gz; do
  sample=$(basename "$r1" _R1.trim.fastq.gz)
  r2="trimmed_reads/fastp/${sample}_R2.trim.fastq.gz"
  unicycler -1 "$r1" -2 "$r2" -o assemblies/unicycler/${sample} -t 4 2>&1 | tee
logs/unicycler_${sample}.log
done
```

Expected outputs: assemblies/unicycler/SAMPLE/assembly.fasta, assembly.gfa and a Unicycler log.

Long-read assembly with Flye

```
mkdir -p assemblies/flye
sample=KPN001
flye --nano-hq raw_reads/${sample}_ONT.fastq.gz --out-dir assemblies/flye/${sample} --threads 4 --
genome-size 5m 2>&1 | tee logs/flye_${sample}.log
```

Expected outputs: assembly.fasta, assembly_graph.gfa, flye.log and assembly_info.txt. Adjust `--genome-size` to approximate species genome size.

Standardise assembly names

```
mkdir -p assemblies/final
for f in assemblies/unicycler/*/assembly.fasta; do
  sample=$(basename $(dirname "$f"))
  cp "$f" assemblies/final/${sample}.fasta
done
```

Expected output: one FASTA per sample in assemblies/final, ready for QUAST, CheckM, AMRFinderPlus and ABRicate.

End-of-module checklist

- Commands ran without unresolved errors.
- Expected output files were created.
- Relevant logs were saved in the logs folder.
- Software and database versions were recorded where applicable.
- Any quality warnings or failed samples were documented.

Module 5: Reference-based Mapping and Variant Calling with BWA, minimap2, SAMtools and BCFtools

Module introduction

Reference-based mapping aligns reads to a known genome so that coverage, consensus differences and variants can be assessed. This approach is useful for SNP analysis, outbreak investigations and detecting mutations when a suitable reference is available. The module introduces standard command-line building blocks for mapping and variant calling.

Key points

- Choose a biologically appropriate reference genome.
- Index the reference before mapping.
- Use BWA-MEM for Illumina reads and minimap2 for ONT reads.
- Sort and index BAM files before downstream analysis.

- Coverage and mapping quality affect confidence in variant calls.

Learning outcomes

- Index a reference genome.
- Map Illumina reads with BWA-MEM.
- Map long reads with minimap2.
- Sort/index BAM files and call variants with BCFtools.

Practical steps

Prepare the reference genome

```
conda activate amr_mapping
cd ~/AMR_training
mkdir -p refs mapping variants logs

# Put your reference genome at refs/reference.fasta
samtools faidx refs/reference.fasta
bwa index refs/reference.fasta
```

Expected outputs: .fai index and BWA index files in refs/.

Map Illumina reads with BWA-MEM

```
for r1 in trimmed_reads/fastp/*_R1.trim.fastq.gz; do
  sample=$(basename "$r1" _R1.trim.fastq.gz)
  r2="trimmed_reads/fastp/${sample}_R2.trim.fastq.gz"
  bwa mem -t 4 refs/reference.fasta "$r1" "$r2"     | samtools sort -@ 4 -o
mapping/${sample}.bwa.sorted.bam
  samtools index mapping/${sample}.bwa.sorted.bam
  samtools flagstat mapping/${sample}.bwa.sorted.bam > mapping/${sample}.bwa.flagstat.txt
done
```

Expected outputs: sorted BAM, BAM index and flagstat summary per sample.

Map ONT reads with minimap2

```
sample=KPN001
minimap2 -ax map-ont -t 4 refs/reference.fasta raw_reads/${sample}_ONT.fastq.gz | samtools sort -@ 4
-o mapping/${sample}.ont.sorted.bam
samtools index mapping/${sample}.ont.sorted.bam
samtools flagstat mapping/${sample}.ont.sorted.bam > mapping/${sample}.ont.flagstat.txt
```

Expected outputs: sorted long-read BAM, BAM index and alignment summary.

Inspect mapping quality and depth

```
sample=KPN001
samtools stats mapping/${sample}.bwa.sorted.bam > mapping/${sample}.bwa.stats.txt
samtools depth -a mapping/${sample}.bwa.sorted.bam > mapping/${sample}.bwa.depth.tsv
awk '{sum+=$3; n++} END {print "Mean depth:", sum/n}' mapping/${sample}.bwa.depth.tsv
```

Expected output: mean depth value. Low coverage can make AMR mutation and SNP calls unreliable.

Call variants with BCFtools

```
for bam in mapping/*.bwa.sorted.bam; do
  sample=$(basename "$bam" .bwa.sorted.bam)
  bcftools mpileup -Ou -f refs/reference.fasta "$bam" | bcftools call -mv -Oz -o
variants/${sample}.vcf.gz
  bcftools index variants/${sample}.vcf.gz
  bcftools stats variants/${sample}.vcf.gz > variants/${sample}.bcftools.stats.txt
  bcftools view variants/${sample}.vcf.gz | head -n 30 > variants/${sample}.vcf.preview.txt
done
```

Expected outputs: compressed VCF, VCF index and stats file per sample.

End-of-module checklist

- Commands ran without unresolved errors.
- Expected output files were created.
- Relevant logs were saved in the logs folder.
- Software and database versions were recorded where applicable.
- Any quality warnings or failed samples were documented.

Module 6: rMAP Pipeline Demonstration

Module introduction

Automated pipelines help standardise multi-step microbial genomics workflows, reduce manual errors and simplify routine surveillance analysis. This module uses rMAP as a demonstration of how individual tools can be combined into a reproducible pipeline. Participants should still understand the underlying steps so they can interpret pipeline outputs critically.

Key points

- Pipelines improve consistency but do not replace expert review.
- Record the exact pipeline version, command and database versions.
- Compare automated outputs with manually generated outputs where possible.
- Inspect QC, assembly, AMR and phylogeny outputs before reporting.
- Pipeline commands may vary by local installation, container or workshop setup.

Learning outcomes

- Explain why automated pipelines are useful in surveillance.
- Recognise typical rMAP inputs and outputs.
- Run or demonstrate rMAP using instructor-provided test data or container instructions.
- Compare pipeline outputs with manually generated outputs.

Practical steps

What rMAP demonstrates

rMAP stands for Rapid Microbial Analysis Pipeline. It was developed to automate bacterial whole-genome analysis steps such as read QC, trimming, assembly, annotation, variant calling, phylogenetic analysis and AMR profiling for bacterial surveillance workflows. In training, rMAP is useful for showing how individual tools can be combined into a reproducible end-to-end workflow.

Prepare a demonstration folder

```
cd ~/AMR_training
mkdir -p rmap_demo/{input,output,logs}

# Copy a small paired-end dataset for demonstration
cp raw_reads/KPN001_R1.fastq.gz rmap_demo/input/
cp raw_reads/KPN001_R2.fastq.gz rmap_demo/input/
```

Expected output: rmap_demo/input contains the selected FASTQ files.

Generic demonstration pattern

Because rMAP deployments may vary by version, institution and container setup, use the official instructor-provided command for your workshop. A typical demonstration should still record the same information:

```
# Example placeholder only - replace with the workshop rMAP command
rmap --help 2>&1 | tee rmap_demo/logs/rmap_help.txt
rmap --input rmap_demo/input --outdir rmap_demo/outputc--threads 4 2>&1 | tee
rmap_demo/logs/rmap_run.log
```

Expected outputs: an output directory containing QC reports, processed reads, assemblies, AMR outputs, phylogeny outputs and/or an integrated report, depending on the rMAP version used.

Interpretation exercise

Participants should answer: Which tools did rMAP run? Which samples passed QC? Which AMR genes were detected? Was the assembly acceptable? Did the pipeline report software and database versions? Which outputs match the manual modules and which differ?

Pipeline caution

Automated pipelines improve reproducibility but do not remove the need for quality review. Users must still inspect QC, database versions, coverage, contamination, reference choice, and phylogenetic assumptions.

End-of-module checklist

- Commands ran without unresolved errors.
- Expected output files were created.
- Relevant logs were saved in the logs folder.
- Software and database versions were recorded where applicable.
- Any quality warnings or failed samples were documented.

Module 7: Assembly QC with QUAST and CheckM

Module introduction

Assembly quality assessment determines whether a genome assembly is suitable for AMR detection, typing, annotation or phylogenetics. QUAST describes contiguity and genome-size statistics, while CheckM estimates completeness and contamination. This module emphasises interpretation and decision-making, not just running commands.

Key points

- QUAST summarises assembly size, contiguity and basic genome statistics.
- CheckM estimates completeness and contamination.
- High contamination or low completeness can invalidate AMR absence calls.
- Unexpected genome size or GC content should trigger review.
- QC decisions should be documented sample by sample.

Learning outcomes

- Run QUAST to assess assembly contiguity and size.
- Run CheckM to estimate completeness and contamination.
- Recognise suspicious assemblies.
- Prepare an assembly QC summary for reporting.

Practical steps

Run QUAST on assemblies

```
conda activate amr_assembly
cd ~/AMR_training
mkdir -p assembly_qc/quast logs

quast.py assemblies/final/*.fasta -o assembly_qc/quast -t 4 2>&1 | tee logs/quast.log
```

Expected outputs: assembly_qc/quast/report.html, report.tsv and transposed_report.tsv.

Interpret QUAST metrics

Important metrics include total length, number of contigs, largest contig, N50, GC content and number of ambiguous bases. For many bacterial isolates, extreme genome size, very high contig count, unusual GC content or many Ns may indicate poor data, contamination, wrong organism, or assembly failure.

Run CheckM lineage workflow

```
conda activate amr_checkm
cd ~/AMR_training
mkdir -p assembly_qc/checkm

checkm lineage_wf -x fasta -t 4 assemblies/final assembly_qc/checkm 2>&1 | tee logs/checkm.log
```

Expected outputs: CheckM result tables and logs in assembly_qc/checkm. CheckM database setup may be required before the first run.

Create a CheckM summary table

```
checkm qa assembly_qc/checkm/lineage.ms assembly_qc/checkm -o 2 -f
assembly_qc/checkm/checkm_summary.tsv --tab_table
```

Expected output: a TSV file with completeness, contamination and strain heterogeneity estimates.

Suggested pass/fail thinking

There is no universal cutoff for every species and purpose. For training, flag assemblies with low completeness, high contamination, abnormal total length, high contig count or unexpected GC content. Do not interpret AMR absence confidently from a poor or incomplete assembly.

End-of-module checklist

- Commands ran without unresolved errors.
- Expected output files were created.
- Relevant logs were saved in the logs folder.
- Software and database versions were recorded where applicable.
- Any quality warnings or failed samples were documented.

Module 8: AMR Detection with AMRFinderPlus and ABRicate

Module introduction

AMR detection tools compare assemblies or sequences against curated resistance databases to identify acquired resistance genes and, in some cases, resistance-associated mutations. This module introduces AMRFinderPlus and ABRicate, highlights the importance of database version reporting, and encourages cautious interpretation of genomic AMR predictions.

Key points

- AMRFinderPlus and ABRicate use databases that must be version-reported.
- Percent identity and coverage affect confidence in AMR gene calls.
- Different databases may produce different gene names or hit lists.
- Genomic prediction is not the same as phenotypic susceptibility testing.
- Report tool version, database version/date and interpretation caveats.

Learning outcomes

- Update and record AMRFinderPlus database information.
- Run AMRFinderPlus on assemblies.
- Run ABRicate against selected databases.
- Compare AMR outputs and prepare surveillance-ready summaries.

Practical steps

AMRFinderPlus database setup and version reporting

```
conda activate amr_amr
cd ~/AMR_training
mkdir -p amr/amrfinder logs db

# Update database when internet is available
Amrfinder -u 2>&1 | tee logs/amrfinder_db_update.log

# Record software/database information
amrfinder --version 2>&1 | tee logs/amrfinder_version.txt
amrfinder --database_version 2>&1 | tee logs/amrfinder_database_version.txt
```

Expected outputs: version files in logs. Include these in the final report.

Run AMRFinderPlus on assemblies

```
for asm in assemblies/final/*.fasta; do
  sample=$(basename "$asm" .fasta)
  amrfinder -n "$asm" -o amr/amrfinder/${sample}.amrfinder.tsv --plus 2>&1 | tee
logs/amrfinder_${sample}.log
done
```

Expected outputs: one TSV per sample with detected AMR genes, classes, subclasses, identity, coverage and database accessions.

ABRicate database setup and reporting

```
abricate --setupdb 2>&1 | tee logs/abricate_setupdb.log
abricate --list > logs/abricate_database_list.tsv
abricate --version 2>&1 | tee logs/abricate_version.txt
```

Expected output: a database list showing available databases and sequence counts.

Run ABRicate with a selected database

```
mkdir -p amr/abricate
for asm in assemblies/final/*.fasta; do
  sample=$(basename "$asm" .fasta)
  abricate --db ncbi "$asm" > amr/abricate/${sample}.abricate_ncbi.tsv
  abricate --db resfinder "$asm" > amr/abricate/${sample}.abricate_resfinder.tsv
done

abricate --summary amr/abricate/*.abricate_ncbi.tsv > amr/abricate/abricate_ncbi_summary.tsv
```

Expected outputs: detailed ABRicate TSV files and a gene-by-sample summary table.

Interpretation and reporting

Report AMR gene name, class, percent identity, coverage, database, database version/date and tool version. Be careful with partial hits, low identity, frameshifted genes, intrinsic resistance, and phenotype-genotype discordance. Genomic AMR predictions support surveillance but should not be treated as clinical susceptibility results without validation.

End-of-module checklist

- Commands ran without unresolved errors.
- Expected output files were created.
- Relevant logs were saved in the logs folder.
- Software and database versions were recorded where applicable.
- Any quality warnings or failed samples were documented.

Module 9: Phylogenetics with BactInspector, Snippy and IQ-TREE

Module introduction

Phylogenetics places bacterial isolates into an evolutionary and epidemiological context. In AMR surveillance, trees can support cluster investigation, transmission hypotheses and comparison of resistant lineages. This module introduces species/reference checking, core SNP calling and maximum-likelihood tree inference while stressing common interpretation limits.

Key points

- Confirm species and reference suitability before SNP phylogenetics.
- Snippy can produce per-sample SNP calls and core alignments.
- IQ-TREE estimates a maximum-likelihood tree and support values.
- A genomic cluster does not automatically prove direct transmission.
- Report reference genome, alignment size, model and support method.

Learning outcomes

- Use BactInspector or instructor-provided species/reference outputs to confirm species and closest reference.
- Run Snippy for reference-based SNP calling.
- Build a core genome alignment.
- Infer a maximum-likelihood tree with IQ-TREE.

Practical steps

Species and closest-reference check with BactInspector

BactInspector uses Mash-based comparisons to support species identification and closest-reference selection. Installation and database setup may be instructor-provided because database preparation can be large.

```
# Example placeholder; replace database path with your workshop database
bactinspector --help 2>&1 | tee logs/bactinspector_help.txt
bactinspector --query assemblies/final/KPN001.fasta --db db/bactinspector --out
phylogeny/bactinspector_KPN001 2>&1 | tee logs/bactinspector_KPN001.log
```

Expected outputs: species/closest-reference result files. Confirm that all isolates belong to the expected species before building a single-species phylogeny.

Run Snippy for each sample

```
conda activate amr_mapping
cd ~/AMR_training
mkdir -p phylogeny/snippy logs

for r1 in trimmed_reads/fastp/*_R1.trim.fastq.gz; do
  sample=$(basename "$r1" _R1.trim.fastq.gz)
  r2="trimmed_reads/fastp/${sample}_R2.trim.fastq.gz"
  snippy --cpus 4 --outdir phylogeny/snippy/${sample} --ref refs/reference.fasta --R1 "$r1" --R2
"$r2" 2>&1 | tee logs/snippy_${sample}.log
done
```

Expected outputs per sample: snps.vcf, snps.tab, snps.aligned.fa, reference-aligned consensus files and logs.

Create a core SNP alignment

```
snippy-core --ref refs/reference.fasta --prefix phylogeny/core phylogeny/snippy/*
```

Expected outputs: phylogeny/core.full.aln, core.aln, core.vcf and related files. The core.aln file is commonly used for phylogenetic inference.

Build a tree with IQ-TREE

```
cd ~/AMR_training
mkdir -p phylogeny/iqtree
```

```
cp phylogeny/core.aln phylogeny/iqtree/core.aln
cd phylogeny/iqtree

iqtree2 -s core.aln -m MFP -B 1000 -T AUTO 2>&1 | tee ../../logs/iqtree_core.log
# If iqtree2 is not available, try: iqtree -s core.aln -m MFP -bb 1000 -nt AUTO
```

Expected outputs: core.aln.treefile, core.aln.iqtree, core.aln.log and support files. The `.treefile` can be viewed in FigTree, iTOL or other tree viewers.

Phylogenetic interpretation cautions

Before interpreting clusters, verify species identity, sample metadata, coverage, core genome size, recombination handling, reference choice and outbreak context. A tree shows genomic relatedness, not automatically transmission. Report methods, reference genome, alignment length, model and support method.

End-of-module checklist

- Commands ran without unresolved errors.
- Expected output files were created.
- Relevant logs were saved in the logs folder.
- Software and database versions were recorded where applicable.
- Any quality warnings or failed samples were documented.